

Introdução

A UML (Unified Modeling Language) é uma linguagem de modelagem amplamente utilizada no desenvolvimento de software, com o objetivo de padronizar a representação visual de sistemas complexos. Antes da UML, o design de software era marcado por uma diversidade de técnicas e notações, o que dificultava a comunicação entre equipes e a padronização de processos. Ao longo desta aula, veremos como a UML foi criada para unificar essas técnicas, facilitando a documentação, o desenvolvimento e a manutenção de projetos. Vamos entender como a UML se aplica tanto no método cascata quanto em metodologias ágeis, adaptando-se a diferentes abordagens de desenvolvimento.

Como era antes da UML?

A evolução do design de software antes do surgimento da UML (Unified Modeling Language) era marcada por um ambiente heterogêneo e despadronizado. Diversas linguagens de modelagem e métodos de desenvolvimento coexistiam, cada um com suas forças e fraquezas, escolhidos conforme as necessidades específicas de cada projeto. Entre as técnicas mais populares estavam a análise de design estruturado, os diagramas de fluxo de dados (DFD), os diagramas entidade-relacionamento (DER) e os diagramas de transição de estados. Esses métodos, apesar de serem amplamente utilizados, refletiam a falta de padronização, o que dificultava a comunicação entre equipes de desenvolvimento e a manutenção de projetos ao longo do tempo.

O diagrama de fluxo de dados, por exemplo, focava exclusivamente no fluxo de dados dentro de um processo ou sistema, representando as entradas e saídas de cada entidade. Criado antes da orientação a objetos, nos anos 1970, ele não abrangia o controle de fluxo, como regras de decisão ou loops,

sendo necessário o uso de fluxogramas para suprir essa lacuna. Os fluxogramas, por sua vez, já eram usados desde 1921 para mapear processos, inclusive antes da existência dos computadores, e complementavam os DFDs ao incluir decisões e ciclos de repetição.

Outro exemplo relevante é o diagrama entidade-relacionamento (DER), amplamente utilizado no design e na depuração de bancos de dados relacionais. Ele representava o relacionamento entre entidades, como pessoas ou objetos, e introduzia conceitos importantes, como entidades fracas, entidades associativas, cardinalidade e atributos. Embora o DER não faça parte da UML, ele continua sendo utilizado, inclusive em conjunto com a UML, mostrando que as técnicas anteriores à UML ainda possuem seu valor.

A ausência de um padrão universal no design de software resultava em um ambiente caótico. A diversidade de técnicas tornava a escolha da abordagem correta desafiadora, gerando ineficiências no desenvolvimento e comunicação entre equipes. Com o tempo, ficou claro que a indústria de software precisava de um padrão que pudesse consolidar essas diferentes técnicas em um modelo mais unificado e acessível.

Foi nesse contexto que a UML surgiu. Ela foi criada com base em técnicas preexistentes, como a modelagem estruturada e os diagramas mencionados, para padronizar o design de software. Seu principal objetivo era facilitar a comunicação entre desenvolvedores, independentemente das metodologias utilizadas, seja o tradicional método cascata ou as metodologias ágeis. Assim, a UML passou a ser um ponto de convergência no desenvolvimento de software, atendendo à necessidade urgente de padronização em um ambiente de extrema diversidade técnica.

Histórico da UML e certificações

A UML (Unified Modeling Language) surgiu como uma resposta à necessidade de padronizar as diversas notações e abordagens que se multiplicavam no campo da modelagem de software. Seu desenvolvimento foi impulsionado pela Rational Software, uma empresa fundada em 1981, com o propósito de criar ferramentas modernas para engenharia de software. A

Rational Software desempenhou um papel central no processo de desenvolvimento da UML entre 1994 e 1995. No entanto, a UML não foi criada do zero. Na verdade, ela resultou da combinação de vários modelos preexistentes, como o OMT (Object Modeling Technique), o Gradebook Method e o OOSE (Object-Oriented Software Engineering). O objetivo principal da criação da UML era justamente unificar essas diferentes técnicas em um padrão acessível e eficiente para a modelagem de software.

A importância da UML cresceu tanto que, em 2003, a Rational Software foi adquirida pela IBM por 2,1 bilhões de dólares, um reflexo do valor que essa ferramenta de suporte ao desenvolvimento de software trazia para a indústria. Com o tempo, a UML tornou-se mais do que uma simples notação; ela passou a ser a linguagem padrão para representar modelos de software, permitindo a visualização, construção e documentação de sistemas de software. Em 1996, foi formado o consórcio UML Partners, e em 1997 surgiu o OMG (Object Management Group), um grupo dedicado a garantir que a UML continue atendendo às necessidades dinâmicas da indústria de software.

A UML evoluiu ao longo dos anos, tornando-se um modelo ISO em 2005 e incorporando novos padrões, como o BPMN (Business Process Model and Notation) em 2006 e 2007, voltado para a modelagem e gestão de processos de negócio. Grandes empresas da indústria tecnológica, como IBM, Microsoft, Oracle, HP e outras, participaram do consórcio que ajudou a estabelecer a UML como o padrão de modelagem mais utilizado na indústria. Seu uso é tão difundido que mesmo desenvolvedores que não trabalham diretamente com a UML acabam utilizando alguns de seus símbolos em seus projetos.

A UML não é uma linguagem de programação, mas uma linguagem de modelagem. Isso significa que seu propósito é permitir a criação de representações visuais dos sistemas de software e de seus artefatos. Além disso, a UML facilita o uso de ferramentas automatizadas para criar diagramas, o que é essencial em grandes projetos. Como exemplo prático, arquitetos de software podem utilizar diagramas UML para documentar as interações entre componentes de um sistema ou para visualizar a arquitetura de uma aplicação. Essa padronização é crucial para garantir a consistência e a clareza na comunicação entre as equipes de desenvolvimento.

Além de sua ampla adoção, a UML oferece diversas certificações para profissionais que desejam se especializar na área, como as certificações UML2 Foundation, Intermediate e Advanced, cada uma mais complexa que a anterior. Essas certificações, oferecidas pelo OMG, exigem a renovação a cada cinco anos, assegurando que o profissional esteja atualizado com as últimas inovações da linguagem. O custo de um exame de certificação gira em torno de 350 dólares, e há uma ampla gama de materiais de estudo disponíveis, incluindo cursos gratuitos e pagos.

Pontos de atenção no uso da UML

Ao utilizar a UML (Unified Modeling Language), é essencial estar ciente de alguns pontos de atenção que podem impactar o sucesso da adoção dessa linguagem no desenvolvimento de software. Um dos primeiros desafios é a resistência de alguns desenvolvedores, principalmente os mais inexperientes, que têm uma inclinação natural para começar diretamente a implementação, sem passar pelo processo de modelagem. Muitos desenvolvedores querem ver o resultado final o mais rápido possível, evitando a fase de planejamento e criação de modelos, o que pode levar a problemas futuros, principalmente em projetos mais complexos que exigem manutenção e escalabilidade ao longo do tempo.

Outro ponto crítico é o uso inadequado de modelagem informal. Muitos desenvolvedores acabam desenhando diagramas de forma improvisada, sem seguir um padrão formal, usando caixas e linhas sem rigor técnico. Embora isso possa parecer uma solução rápida, o uso de diagramas informais pode comprometer a comunicação entre os membros da equipe e dificultar a continuidade do projeto. A UML, por ser uma linguagem formal, proporciona diagramas que seguem um padrão reconhecido, facilitando a colaboração e o entendimento entre os envolvidos no desenvolvimento do sistema.

Estima-se que menos de 9% dos projetos utilizem exclusivamente diagramas UML. Isso significa que, em muitos casos, há uma mistura de diagramas formais da UML com outras notações, formais ou informais, o que pode gerar confusão e atrapalhar a comunicação entre os stakeholders. A utilização de diferentes padrões de modelagem pode causar inconsistências,

especialmente quando as equipes tentam integrar esses diferentes modelos em um único projeto.

Além disso, há a questão da curva de aprendizado. Para utilizar a UML de forma eficiente, todos os envolvidos precisam aprender a linguagem, o que pode demandar tempo e dedicação. Aprender a usar ferramentas que suportam a criação de diagramas UML, como o AstaUML, por exemplo, é apenas parte do processo. O desafio maior é garantir que o uso da notação esteja de acordo com as regras da UML, garantindo que o sistema seja corretamente modelado, independentemente da ferramenta utilizada.

Por fim, outro ponto de atenção relevante é a manutenção dos diagramas ao longo do ciclo de vida do software. Mesmo que os diagramas UML sejam criados adequadamente durante o desenvolvimento, muitas vezes eles não são atualizados quando o software passa por modificações ou novas funcionalidades são adicionadas. Isso pode resultar em uma discrepância entre o sistema real e sua documentação, comprometendo a compreensão dos novos integrantes da equipe e a manutenção do projeto. Manter os diagramas atualizados é fundamental para garantir a consistência e a eficácia da documentação ao longo do tempo.

UML e o método cascata

O método cascata, conhecido também como Waterfall, é um dos métodos mais tradicionais de desenvolvimento de software, caracterizado por suas fases bem definidas e sequenciais. Nele, cada etapa do desenvolvimento ocorre de maneira linear, o que significa que o progresso de uma fase para a outra só acontece quando a fase anterior é finalizada completamente. Esse processo rigoroso e estruturado segue um fluxo em que não há possibilidade de “pular” fases ou retroceder para fases anteriores, garantindo que cada etapa seja concluída antes que a próxima comece. As fases incluem a análise de requisitos, projeto, implementação, testes, integração e manutenção, todas ocorrendo uma após a outra.

No contexto de projetos com escopo bem definido, como licitações públicas em que os requisitos do software são especificados de forma clara e detalhada, o método cascata se adapta bem. A ênfase na documentação em cada fase permite que o conhecimento adquirido durante o desenvolvimento seja preservado, mesmo que haja troca de desenvolvedores ao longo do projeto. Isso é especialmente importante quando o turnover de equipes é alto, pois a documentação clara de cada fase assegura que o novo time possa retomar o projeto sem perda significativa de informação.

Contudo, o método cascata apresenta algumas limitações. Uma delas é sua inflexibilidade em relação à revisão dos requisitos e do design. Como cada fase é finalizada antes do início da próxima, não há espaço para voltar atrás e revisar os requisitos ou o design, o que pode ser problemático se o cliente não tiver definido completamente suas necessidades no início do projeto. Em muitos casos, os clientes percebem, posteriormente, que alguns requisitos importantes não foram comunicados corretamente ou que o escopo precisa ser ajustado. Nessas situações, o método cascata não lida bem com mudanças, uma vez que o desenvolvimento já avançou para fases posteriores.

Outro ponto importante é o custo de correção de erros. No método cascata, os erros introduzidos nas primeiras etapas, como na fase de requisitos ou projeto, são mais fáceis e baratos de corrigir do que erros cometidos nas fases finais, como na implementação. Corrigir um erro de requisitos após a implementação pode ser extremamente caro, pois isso exigiria revisar e alterar não apenas o código, mas também a documentação e o design do sistema.

A UML desempenha um papel fundamental no contexto do método cascata, pois permite modelar o sistema nas fases iniciais do desenvolvimento, facilitando a criação de uma documentação clara e precisa. Esses modelos são úteis não apenas para a fase de desenvolvimento, mas também para as futuras manutenções e atualizações do sistema. A modelagem com UML garante que, desde o início do projeto, todos os aspectos estruturais e comportamentais do software estejam bem representados, proporcionando uma comunicação eficiente entre todos os stakeholders ao longo do ciclo de vida do projeto.

UML e metodologias ágeis

O desenvolvimento ágil de software surgiu como uma resposta ao peso excessivo que os processos tradicionais, como o método cascata, traziam ao desenvolvimento. Nas décadas de 50 e 60, os profissionais de software eram “heróis” que dominavam todos os aspectos do ciclo de desenvolvimento, desde a coleta de requisitos até a implementação e suporte. No entanto, com o aumento significativo do número de programadores nas décadas de 70 e 80, muitos desses profissionais experientes estavam se aposentando, e a nova geração de desenvolvedores ainda estava em processo de aprendizado. Assim, foi necessário repensar as metodologias, o que culminou na criação do Manifesto Ágil em 2001.

O Manifesto Ágil foi resultado do trabalho de 17 profissionais que perceberam que o desenvolvimento de software estava se tornando muito burocrático, principalmente devido à ênfase excessiva em documentação e processos rigorosos. O manifesto trouxe quatro valores principais: indivíduos e interações acima de processos e ferramentas, funcionamento do software acima de documentação abrangente, colaboração com o cliente acima de negociação contratual, e resposta a mudanças acima de seguir um plano. Esses valores estabeleceram as bases para as metodologias ágeis, como Scrum, XP e Lean Development.

Uma dúvida comum é se a UML pode ser utilizada em metodologias ágeis, já que muitos associam essa linguagem apenas ao método cascata. A resposta é sim, a UML pode ser utilizada em ambientes ágeis, mas com alguns pontos de atenção. Em metodologias ágeis, as informações necessárias para modelar o sistema nem sempre estão disponíveis no início do projeto, o que dificulta a criação de um design antecipado. Além disso, devido às constantes iterações e mudanças, é necessário atualizar frequentemente os diagramas UML para refletir as evoluções do projeto, o que pode ser um desafio, especialmente considerando que muitos desenvolvedores não gostam de atualizar diagramas.

Os benefícios da UML em metodologias ágeis incluem suporte ao desenvolvimento futuro, geração de documentação clara e compreensível, definição da arquitetura do sistema e facilitação da comunicação entre membros da equipe. Um exemplo prático de uso da UML em ambientes ágeis é o uso de diagramas de classes para definir a estrutura inicial do sistema e diagramas de sequência para documentar o comportamento dinâmico durante as sprints. Esses modelos ajudam a equipe a manter uma visão clara do projeto, mesmo com as mudanças frequentes.

No entanto, é importante lembrar que o foco das metodologias ágeis é o desenvolvimento do software em si, e não a criação de modelos detalhados. A UML, nesse contexto, deve ser utilizada como uma ferramenta de suporte ao desenvolvimento, não como um fim em si mesma. Se todo o tempo for gasto em modelagem, o time perde agilidade. Portanto, os diagramas devem ser simples e eficientes, refletindo as necessidades do projeto sem gerar sobrecarga.

Concluimos que a UML desempenha um papel crucial no desenvolvimento de software, proporcionando uma linguagem padronizada que facilita a comunicação entre as equipes e a documentação dos sistemas. Ao longo desta aula, exploramos o histórico da UML, sua criação e como ela unifica diversas técnicas de modelagem. Também discutimos os pontos de atenção ao utilizá-la e sua flexibilidade para ser aplicada tanto no método cascata quanto em metodologias ágeis. A UML, portanto, se consolida como uma ferramenta essencial para garantir a clareza e a eficiência no desenvolvimento de projetos de software.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Modelo em Cascata - Ciclos de Vida de Desenvolvimento de Software”, que está disponível no canal Bóson Treinamentos no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamentos da programação de computadores**. 3.ed. Pearson: 2012.

BRAGA, P. H. **Teste de software**. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). **Arquitetura de software**. Pearson: 2017.

GALLOTTI, G. M. A. **Qualidade de software**. Pearson: 2015.

MEDEIROS, E. **Desenvolvendo software com UML 2.0 definitivo**. Pearson: 2004.

MORAIS, I. S. de. (Org.). **Engenharia de software**. Pearson: 2017.

PFLEEGER, S. L. **Engenharia de software: teoria e prática**. 2.ed. Pearson: 2003.

SOMMERVILLE, I. **Engenharia de software**. 10.ed. Pearson: 2019.